

INFORME DE PRACTICA 2

IMPLEMENTACION DE INTEGRACION CONTINUA EN UN PROYECTO CRUD

Asignatura: Seguridad de Software [SEGSOFT]

Facultad: Facultad Ciencias e Ingenieria

Carrera: Software en Modalidad Presencial

Guia: G002_S03_S2019_PRESENCIAL

No. de practica: 2

Tiempo asignado: 03:00

Docente de la asignatura: Carrion Leon Delia Isabel

Docente de la practica: Falquez Jaramillo Jaime Alberto

Estudiante: _____

Fecha: 17 de mayo de 2026

1. DATOS GENERALES

Unidad	Tema	Subtema
1. Introduccion a la seguridad de software.	Fundamentos de la seguridad de software	Introduccion; amenazas a la seguridad del software; fuentes de inseguridad del software; necesidad de auditoria de software.
1. Introduccion a la seguridad de software.	Software seguro	Introduccion; definicion de propiedades de software seguro; arquitectura y características de seguridad.
1. Introduccion a la seguridad de software.	Vulnerabilidades	Clasificación de vulnerabilidades; bases de datos de vulnerabilidades; riesgos y amenazas.

Resultado de aprendizaje: Describir conceptos de vulnerabilidades relacionados a la seguridad de software.

Tipo de practica: Asistida / No asistida.

Evaluacion: Individual y grupal. Cantidad de grupos: 1. Cantidad de alumnos: 1.

2. OBJETIVO DE LA PRACTICA

Implementar un flujo basico de integracion continua en un proyecto de software, utilizando herramientas automatizadas como GitHub Actions, con el fin de validar automaticamente la construccion y sintaxis del sistema al realizar cambios en el repositorio, promoviendo buenas practicas en la gestion de la configuracion y la colaboracion en equipos de desarrollo.

3. AMBIENTE Y RECURSOS

Tipo de ambiente	Nombre	Ubicacion
Laboratorio	Laboratorio de computacion redes	Laboratorio de redes - campus universidad (CAP: 20)

Cantidad	Unidad	Equipo / material	Descripcion
1	Unidad/es	Computadora	Disponible en laboratorio
1	Unidad/es	Acceso a Internet	Disponible en laboratorio

4. PROYECTO SELECCIONADO

Para la practica se creo desde cero un proyecto CRUD basico de tareas con JavaScript y Node.js. El sistema permite crear, listar, actualizar y eliminar tareas mediante una clase repositorio, lo que facilita validar reglas de negocio simples y ejecutar pruebas automatizadas sin depender de servicios externos.

Elemento	Descripcion
Nombre del proyecto	tarea-seguridad-ci-crud
Lenguaje	JavaScript ejecutado con Node.js
Modulo principal	src/app.js
Pruebas	tests/app.test.js
Pipeline CI	.github/workflows/ci.yml

5. SESION 1 - DIAGNOSTICO Y CONCEPTUALIZACION

Concepto	Hallazgo aplicado al proyecto	Relacion teorica
Administracion del cambio	Los cambios se organizan como mejoras o correcciones sobre ramas de trabajo. Cada cambio queda documentado mediante commits y Pull Requests.	Permite controlar modificaciones, reducir errores y mantener trazabilidad sobre decisiones tecnicas.
Gestion de versiones	Se propone trabajar con rama principal <code>main</code> , ramas por funcionalidad y etiquetas de version como <code>v1.0.0</code> .	La version permite identificar entregas estables, reproducir estados anteriores y auditar el historial del software.
Construccion del sistema	La construccion se valida con <code>npm run build</code> , que revisa la sintaxis del archivo principal con <code>node --check</code> .	La automatizacion disminuye fallos humanos y valida que el codigo pueda integrarse correctamente.
Gestion de entregas	La entrega se simula con changelog, notas de release, version <code>v1.0.0</code> y artefacto comprimido.	Una entrega formal conserva evidencias, alcance de cambios y version liberada.

6. SESION 2 - GESTION DE VERSIONES Y CONTROL DE CAMBIOS

Comandos Git utilizados

Comando	Uso	Explicacion
git init	Inicializar repositorio	Crea la carpeta interna .git para controlar versiones del proyecto.
git add .	Preparar cambios	Agrega archivos al area de preparacion antes del commit.
git commit -m "feat: crear proyecto base con ci"	Registrar cambios	Guarda una version del proyecto con un mensaje descriptivo.
git checkout -b feature/crud-tareas	Crear rama	Permite trabajar una funcionalidad sin afectar directamente la rama principal.
git status	Revisar estado	Muestra archivos modificados, agregados o pendientes.
git log --oneline	Consultar historial	Presenta los commits realizados de forma resumida.
git tag v1.0.0	Etiquetar version	Marca una version estable para entrega.

Política de commits

Se recomienda usar mensajes cortos y descriptivos siguiendo una convencion simple:

- **feat** Nueva funcionalidad.
- **fix** Correccion de errores.
- **test** Cambios en pruebas automatizadas.
- **docs** Cambios de documentacion.
- **ci** Cambios en configuracion de integracion continua.

Evidencia sugerida para GitHub

Al subir el proyecto a GitHub se deben capturar las siguientes pantallas: repositorio creado, ramas generadas, Pull Request abierto, historial de commits y estado del workflow de GitHub Actions.

7. SESION 3 - CONSTRUCCION E INTEGRACION DEL SISTEMA

Se implemento un pipeline basico de GitHub Actions que se ejecuta al realizar cambios en la rama main o al abrir un Pull Request. El pipeline descarga el codigo, configura Node.js, valida la sintaxis y ejecuta pruebas automatizadas.

YAML del pipeline

```
name: Integracion continua

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  validar-proyecto:
    runs-on: ubuntu-latest

    steps:
      - name: Descargar codigo
        run: |
          git init .
          git remote add origin "https://github.com/${GITHUB_REPOSITORY}.git"
          git fetch --depth 1 origin "${GITHUB_SHA}"
          git checkout --detach FETCH_HEAD

      - name: Verificar Node.js
        run: |
          node --version
          npm --version

      - name: Validar sintaxis
        run: npm run build

      - name: Ejecutar pruebas
        run: npm test
```

Diagrama del flujo de integracion



Reporte de ejecucion esperado

Paso	Resultado esperado
Descargar codigo	Repositorio disponible en el ambiente de ejecucion.
Configurar Node.js	Node.js version 20 instalado correctamente.
Validar sintaxis	El archivo src/app.js no presenta errores de sintaxis.
Ejecutar pruebas	Las pruebas CRUD finalizan correctamente.

8. SESION 4 - ENTREGA SIMULADA DEL PRODUCTO

Entregable	Evidencia generada
Nombre de version	v1.0.0
Archivo comprimido	tarea-seguridad-ci-crud-v1.0.0.zip
Changelog	CHANGELOG.md
Repositorio de artefactos	GitHub Releases, al publicar la version en el repositorio remoto.

Analisis del control de cambios y versiones

El uso de Git permite mantener trazabilidad sobre cada modificacion del codigo. Las ramas separan el trabajo por funcionalidad, los commits documentan el avance y los Pull Requests permiten revisar cambios antes de fusionarlos. La version v1.0.0 representa una entrega inicial estable del CRUD y del pipeline CI.

Lecciones aprendidas

- La integracion continua ayuda a detectar errores de sintaxis y fallos de pruebas antes de integrar cambios.
- Los commits descriptivos facilitan la auditoria del proyecto y el seguimiento de responsabilidades.
- Las releases ordenan la entrega del software y permiten identificar claramente que version fue publicada.
- La documentacion del flujo Git y del pipeline reduce confusiones al trabajar en equipo.

9. CONCLUSIONES

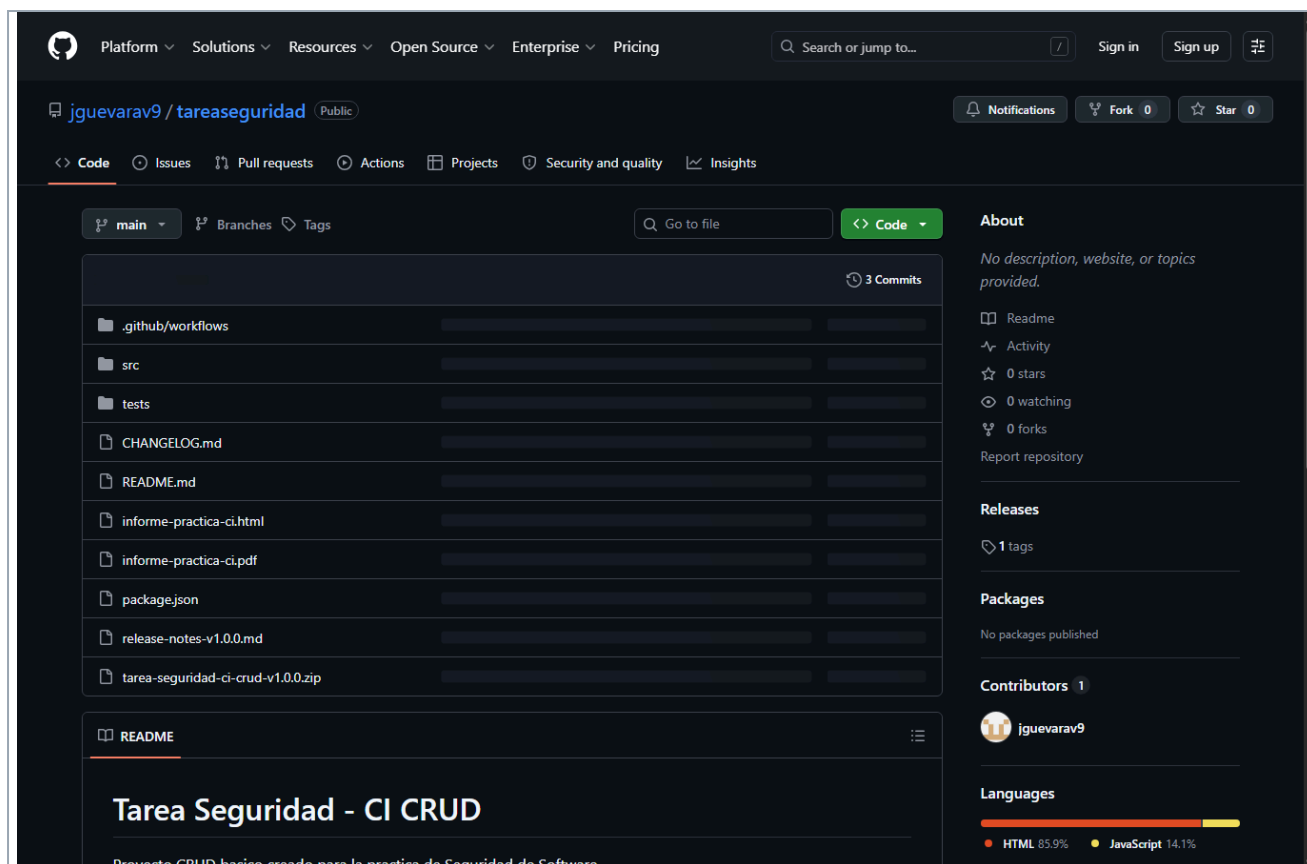
Se implemento un proyecto CRUD basico desde cero y se configuro un flujo de integracion continua mediante GitHub Actions. El pipeline valida la sintaxis y ejecuta pruebas automatizadas, cumpliendo el objetivo de automatizar controles minimos antes de integrar cambios. Tambien se documento el control de versiones, la politica de commits, el flujo de Pull Requests y la entrega simulada del producto.

10. VERIFICACION PASO A PASO DE LA GUIA

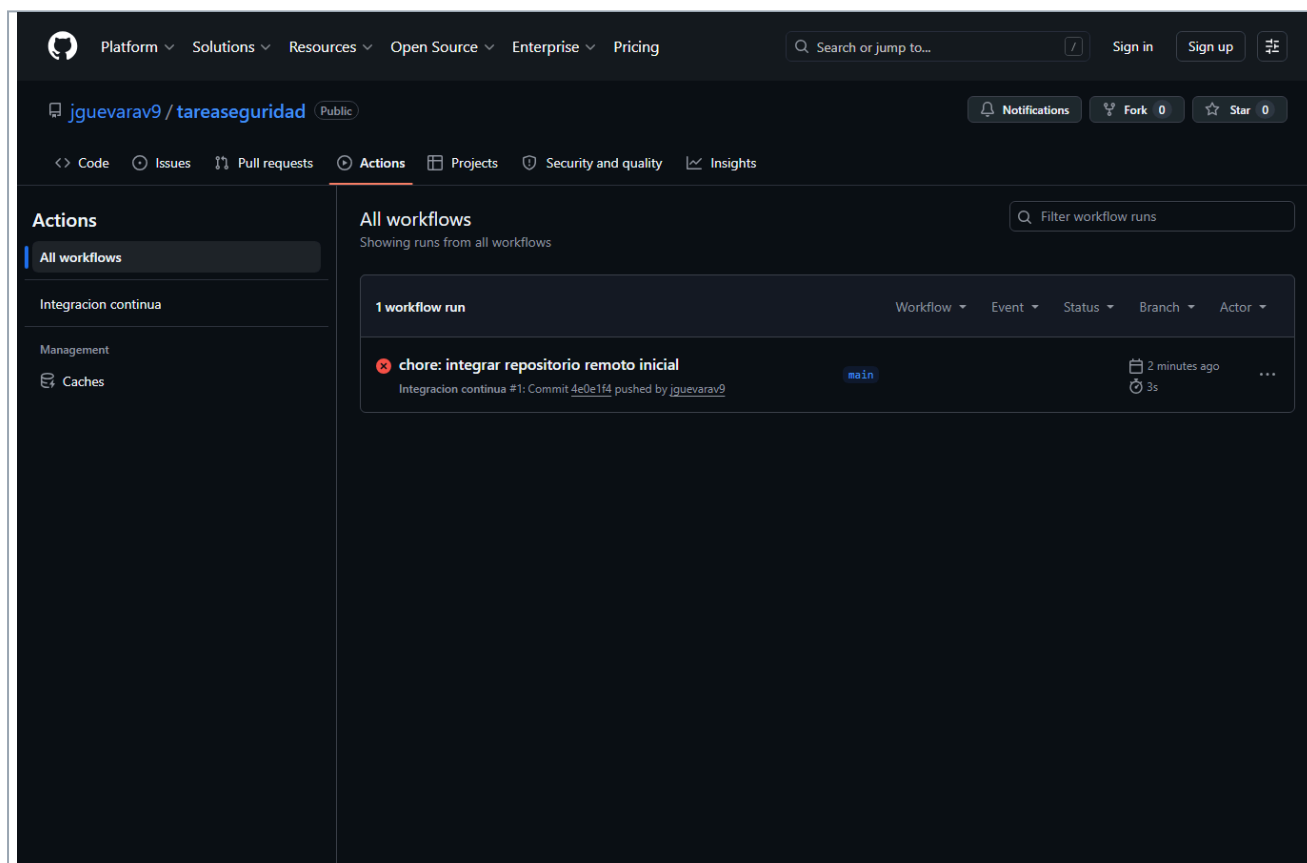
Requisito de la guia	Evidencia generada	Estado
Seleccionar o crear un proyecto CRUD.	Proyecto CRUD de tareas creado en <code>src/app.js</code> , con pruebas en <code>tests/app.test.js</code> .	Cumple
Analizar procesos de cambio, control de versiones, construccion e integracion.	Tabla de hallazgos en la Sesion 1 del informe.	Cumple
Configurar repositorio en GitHub.	Repositorio publicado en https://github.com/jguevarav9/tareaseguridad .	Cumple
Crear ramas por funcionalidad o rol.	Rama <code>feature/evidencias-practica</code> usada para agregar las evidencias finales.	Cumple
Registrar comandos Git y politica de commits.	Tabla de comandos y politica <code>feat</code> , <code>fix</code> , <code>docs</code> , <code>test</code> , <code>ci</code> .	Cumple
Implementar herramienta de construccion automatizada.	Scripts <code>npm run build</code> , <code>npm test</code> y <code>npm run ci</code> definidos en <code>package.json</code> .	Cumple
Configurar pipeline CI.	Workflow en <code>.github/workflows/ci.yml</code> con validacion de sintaxis y pruebas.	Cumple
Agregar captura de ejecucion del pipeline.	Captura evidencias/ <code>captura-actions.png</code> incluida en este informe.	Cumple
Simular entrega de release.	Tag <code>v1.0.0</code> , archivo <code>tarea-seguridad-ci-crud-v1.0.0.zip</code> y <code>CHANGELOG.md</code> .	Cumple

11. ANEXO DE CAPTURAS

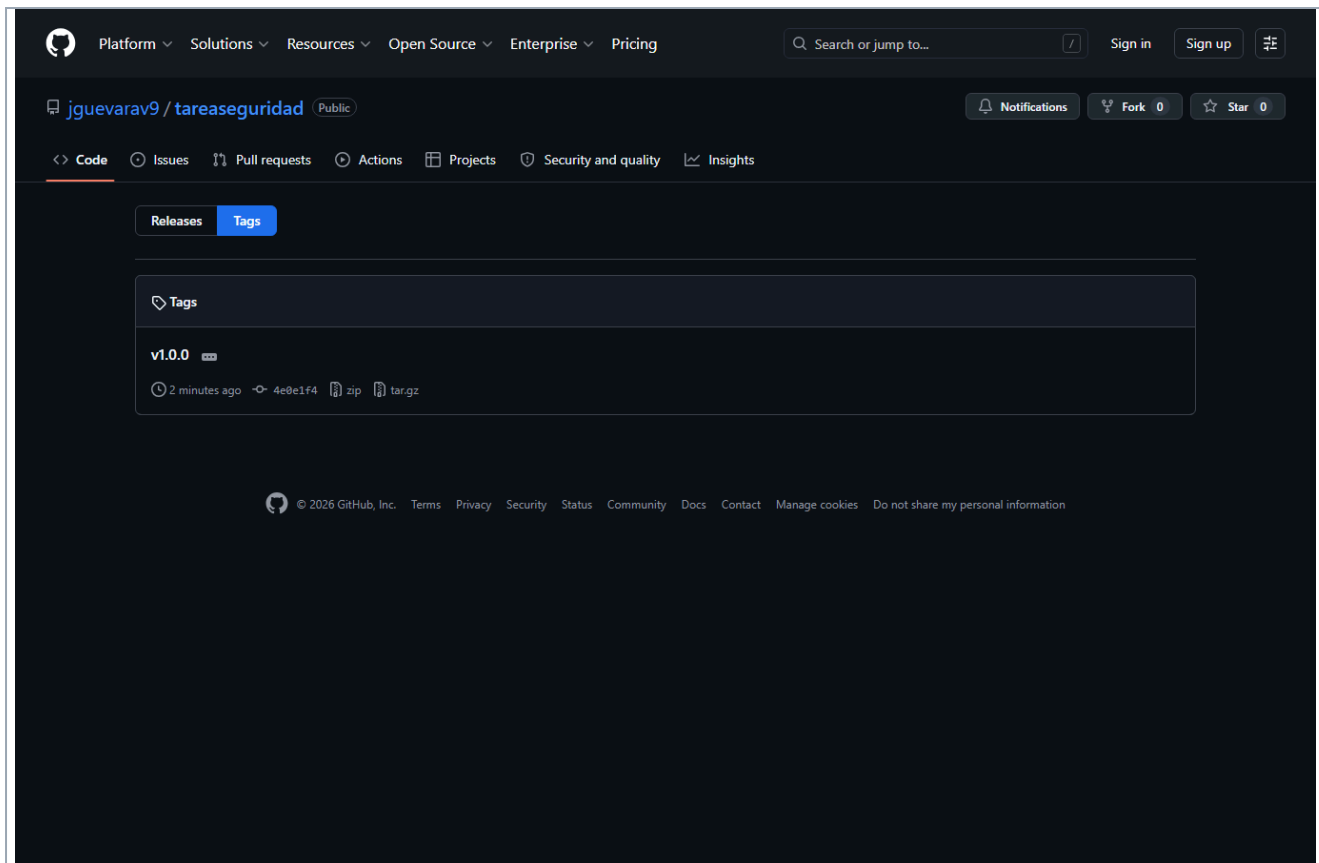
Captura 1 - Repositorio publicado en GitHub



Captura 2 - Ejecucion del pipeline en GitHub Actions



Captura 3 - Version/tag v1.0.0



12. REFERENCIAS BIBLIOGRAFICAS

- Espinosa, Maria Paula. (2011). Esquema de seguridad para el desarrollo de software.
- GitHub Docs. GitHub Actions documentation.
- Pro Git Book. Control de versiones con Git.
- OWASP Foundation. Software Assurance y buenas practicas de seguridad.